

WAOW Admin — ຄູ່ມືໂຄງສ້າງ Backend

ສໍາລັບນຳສະເໜີ / ສອບພາດ

Repository: <https://github.com/souksanDev1/waow-admin-task>

ອະທິບາຍໄຟລເດີ ແລະ ການເຮັດວຽກຂອງໂຄດ Backend ເປັນພາສາລາວ

1. Backend ແມ່ນຫຍັງ

Backend ນີ້ແມ່ນ REST Admin API ທີ່ສ້າງດ້ວຍ:

- Node.js + Express — ເຊີບເວີ HTTP
- PostgreSQL + Sequelize — ຖານຂໍ້ມູນ ແລະ ORM
- JWT — ຢືນຢັນຕົວຕົນ admin
- RBAC — ຄວບຄຸມສິດຕາມ role (permissions ແບບ JSON ແບບເຄື່ອນໄຫວ)
- npm — package manager

ພາທ Base ຂອງ API: `/api/v1`

ພອດເລີ່ມຕົ້ນ: `3002`

ຖານຂໍ້ມູນຮ່ວມກັບ User API: Postgres ທີ່ `localhost:5434` , ຊື່ DB `waow_backend`

ຟັງຊັນຫຼັກ

1. Admin login (username + password → JWT)
 2. ຈັດການ Role (permissions ແບບ dynamic)
 3. Admin CRUD ພ້ອມ **soft delete**
 4. Reset password (ສະເພາະ SUPER_ADMIN)
 5. ລາຍການ users ຈາກຕາຕະລາງ `users` ທີ່ແຊຣ໌ (ອ່ານຢ່າງດຽວ)
-

2. ການໄຫຼຂອງ Request (ສໍາຄັນທີ່ສຸດ ຄວນຈື່)

ທຸກ request ທີ່ຕ້ອງ auth ຈະໄຫຼແບບນີ້:

Client (Frontend / Postman)

- routes (URL + HTTP method)
- middlewares (validate, JWT, RBAC)
- controllers (ຊັ້ນ HTTP ບາງໆ)
- services (ທຸລະກິດ / business logic)
- models (Sequelize ↔ PostgreSQL)
- response JSON { error, code, message, data }

ຖ້າຜິດພາດ, `errorHandler` ຈະແປງເປັນ error envelope ມາດຕະຖານ

ເຄັດສອບພາດ: ບອກວ່າ controller ບາງ; **service** ຄຸ້ມກົດຫຼະກົດ ແລະ transaction

3. ໄຟລເດີລະດັບ `backend/`

ເລີ່ມທາງ	ຈຸດປະສົງ
<code>package.json</code>	ສະຄຣິບ (<code>dev</code> , <code>db:migrate</code> , <code>db:seed</code> , <code>lint</code>) ແລະ dependencies
<code>.env</code> / <code>.env.example</code>	ພອດ, JWT secret, ການເຊື່ອມຕໍ່ DB (ບໍ່ commit ໄຟລ <code>.env</code>)
<code>.sequelizerc</code>	ບອກ sequelize-cli ວ່າ config / models / migrations / seeders ຢູ່ໃສ
<code>eslint.config.js</code>	ກົດ lint
<code>src/</code>	ໂຄດທັງໝົດຂອງແອັບ

4. ຈຸດເລີ່ມຕົ້ນຂອງແອັບ

`src/server.js`

- ໂຫຼດ Express app
- ເຊື່ອມຕໍ່ DB ດ້ວຍ `sequelize.authenticate()`
- ເປີດ listen ທີ່ `PORT` (3002)

ນີ້ແມ່ນສິ່ງທີ່ `pnpm dev` / `nodemon` ຮັບ

`src/app.js`

- ສ້າງ Express application
- ເປີດ `CORS`, ອ່ານ JSON body
- mount route ທັງໝົດພາຍໃຕ້ `/api/v1`
- ຈັດການ route ທີ່ບໍ່ພົບ (404)
- ຜິດ `errorHandler` ທົ່ວໂລກ

5. `src/config/` – ການຕັ້ງຄ່າ

ໄຟລ	ຫນ້າທີ່
-----	---------

env.js	ອ່ານ env (port, JWT, DB) ໃຫ້ສ່ວນອື່ນໃຊ້
database.js	ການຕັ້ງຄ່າເຊື່ອມຕໍ່ Sequelize / sequelize-cli

ເກັບຄ່າສະພາບແວດລ້ອມໄວ້ບ່ອນດຽວ ບໍ່ປະປົນໃນ business code

6. src/routes/ – ແຜນທີ່ URL

Routes ຕອບຄໍາຖາມ: URL ໃດໄປໃສ ແລະ ໃຊ້ middleware ໃດ

ໄຟລ	ໜ້າທີ່
index.js	ລວມ router ທັງໝົດ + GET /health
auth.routes.js	POST /auth/login (ສາທາລະນະ)
role.routes.js	/roles – ຕ້ອງ JWT + SUPER_ADMIN
admin.routes.js	/admins – ຕ້ອງ JWT + ສິດ admin_module
user.routes.js	GET /users – ຕ້ອງ JWT + user_module.can_view

Endpoint ສໍາຄັນ

Method	Path	ສິດເຂົ້າເຖິງ
POST	/api/v1/auth/login	ສາທາລະນະ
CRUD	/api/v1/roles	SUPER_ADMIN
CRUD	/api/v1/admins	ຕາມ permission
POST	/api/v1/admins/:id/reset-password	SUPER_ADMIN
GET	/api/v1/users	user_module.can_view
GET	/api/v1/health	ສາທາລະນະ

7. src/middlewares/ – ປະຕູຄວາມປອດໄພ ແລະ ກວດຂໍ້ມູນ

ໄຟລ	ເຮັດຫຍັງ
validate.js	ກວດ body ດ້ວຍ Joi; ປະຕິເສດຟົວທີ່ບໍ່ຮູ້ຈັກ (VALIDATION_ERR_UNKNOWN_FIELD)

authenticate.js	ອ່ານ Authorization: Bearer <token>, ຍືນຢັນ JWT, ໂຫຼດ admin + role ໃສ່ req.auth
authorize.js	requireSuperAdmin ແລະ requirePermission(module, action) ສໍາລັບ RBAC
errorHandler.js	ແປງ error ເປັນ { error: true, code, message, data }

ເຄັດສອບພາດ:

Authentication = “ເຈົ້າແມ່ນໃຜ?”

Authorization = “ເຈົ້າເຮັດຫຍັງໄດ້ບໍ?”

8. src/validations/ – ກົດຂໍ້ມູນເຂົ້າ

schemas.js

Schema Joi ສໍາລັບ:

- Login (username , password)
- ສ້າງ / ແກ້ role (name , ຮູບແບບ permissions)
- ສ້າງ / ແກ້ admin
- Reset password

ທຸກ schema ໃຊ້ .unknown(false) ເພື່ອປະຕິເສດພົວເກີນ (ຕາມເອກະສານວຽກ)

9. src/controllers/ – ຊັ້ນ HTTP

Controller ເຮັດພຽງ:

1. ອ່ານ req.body / req.params / req.auth
2. ເອີ້ນ service ທີ່ກົງກັນ
3. ສົ່ງຄືນ success(res, data, message)

ບໍ່ໃສ່ logic ທຸລະກິດຫນັກ

ໄຟລ	ຈັດການ
auth.controller.js	Login
role.controller.js	ລາຍການ / ສ້າງ / ແກ້ / ລຶບ role
admin.controller.js	CRUD admin + soft delete + reset password

10. `src/services/` – Business logic (ຫຼັກຂອງລະບົບ)

ໄຟລເດີນີ້ສໍາຄັນທີ່ສຸດເມື່ອອະທິບາຍໂປຣເຈັກ

ໄຟລ	ໜ້າທີ່
<code>auth.service.js</code>	ຫາ admin ທີ່ບໍ່ຖືກ soft-delete, ທຽບລະຫັດ, ອອກ JWT, ສົ່ງ profile + permissions
<code>role.service.js</code>	CRUD role; ປຶກເບິ່ງ role ລະບົບ SUPER_ADMIN / NORMAL; ຫ້າມລຶບຖ້າຍັງມີ admin ໃຊ້ຢູ່
<code>admin.service.js</code>	CRUD admin (ໃຊ້ transaction ເມື່ອຈໍາເປັນ); soft delete; reset password
<code>user.service.js</code>	ອ່ານຕາຕະລາງ users ຢ່າງດຽວ (ແຊຣ໌ກັບ User API)

Soft delete (admins)

ເມື່ອລຶບ admin:

- ຕັ້ງ `is_deleted = true`
- ຕັ້ງ `deleted_at = now`
- ບໍ່ລຶບແຖວອອກຈາກຖານຂໍ້ມູນຖາວອນ
- admin ທີ່ soft-delete ແລ້ວ login ບໍ່ໄດ້ ແລະ ບໍ່ຂຶ້ນໃນລາຍການປຶກກະຕິ

Transaction

ການສ້າງ/ແກ້ admin ໃຊ້ Sequelize transaction ເພື່ອກັນຂໍ້ມູນຄ້າງຄຶງເຄິ່ງໆ

11. `src/models/` – ແຜນທີ່ຕາຕະລາງ

ໄຟລ	ຕາຕະລາງ / ຄວາມໝາຍ
<code>index.js</code>	ໂຫຼດ model, ຜູກ associate, export sequelize
<code>user.js</code>	users (ແຊຣ໌ກັບ User API; ອ່ານຢ່າງດຽວໃນວຽກນີ້)
<code>role.js</code>	roles ພ້ອມ JSONB permissions
<code>admin.js</code>	admins ພ້ອມພົວ soft-delete + <code>toSafeJSON()</code> (ບໍ່ສົ່ງ password hash)

ຄວາມສໍາພັນ

- `Admin.belongsTo(Role)`
- `Role.hasMany(Admin)`

ຕົວຢ່າງ Permissions JSON

```
{
  "admin_module": {
    "can_create": false,
    "can_delete": true,
    "can_update": true,
    "can_view": true
  },
  "user_module": {
    "can_create": true,
    "can_delete": false,
    "can_update": true,
    "can_view": true
  }
}
```

12. `src/migrations/` – ສ້າງ/ປັບໂຄງຕາຕະລາງ

ຮັບດ້ວຍ: `pnpm db:migrate`

Migration	ຜົນ
<code>...create-users.js</code>	ສ້າງ users ເມື່ອຍັງບໍ່ມີ (ຂ້າມຖ້າ User API ສ້າງໄວ້ແລ້ວ)
<code>...create-roles.js</code>	ສ້າງ roles
<code>...create-admins.js</code>	ສ້າງ admins ພ້ອມ FK ໄປ roles

ອະນຸຍາດໃຫ້ຕອບເງື່ອນໄຂ: ໃຊ້ຖານຂໍ້ມູນດຽວກັນກັບໂປຣເຈັກ User API

13. `src/seaders/` – ຂໍ້ມູນເລີ່ມຕົ້ນ

`default-roles-and-superadmin.js`

ຮັບດ້ວຍ: `pnpm db:seed`

ສ້າງ (ຖ້າຍັງບໍ່ມີ):

1. Role `SUPER_ADMIN` – ສິດເຕັມ
 2. Role `NORMAL` – ເບິ່ງເທົ່ານັ້ນ (ບໍ່ສ້າງ/ແກ້/ລຶບ)
 3. Admin `superadmin` / `123456` (hash ດ້ວຍ bcrypt) ຜູກ `SUPER_ADMIN`
-

14. `src/utlis/` – ເຄື່ອງມືຮ່ວມ

ໄຟລ	ໜ້າທີ່
<code>response.js</code>	ຮູບແບບ JSON success / fail ມາດຕະຖານ
<code>errors.js</code>	AppError ພ້ອມ status + error code
<code>jwt.js</code>	ເຊັນ ແລະ ກວດ JWT
<code>password.js</code>	hash / ທຽບລະຫັດດ້ວຍ bcrypt

ຮູບແບບ Response (ຕາມເອກະສານວຽກ)

ສໍາເລັດ:

```
{
  "error": false,
  "code": 0,
  "message": "Success",
  "data": {}
}
```

ລົ້ມເຫຼວ:

```
{
  "error": true,
  "code": "ADMIN_ERR_001",
  "message": "Error message here",
}
```

```
"data": {}  
}
```

15. ສະຄຣິບອະທິບາຍ 60 ວິນາທີ

“Backend ໃຊ້ Express ແບບ clean architecture.

Routes ກຳນົດ URL. Middlewares ຈັດການ validation, JWT authentication ແລະ RBAC authorization.

Controllers ບາງ. Services ມີ logic ທຸລະກິດ ເຊັ່ນ soft delete ແລະ transaction.

Models ແຜນທີ່ໄປຕາຕະລາງ PostgreSQL `users`, `roles`, ແລະ `admins`.

ພວກເຮົາໃຊ້ຖານຂໍ້ມູນດຽວກັບ User API: ໃຊ້ `users` ຊື່ ແລະ ເພີ່ມຕາຕະລາງ admin/role.

ມີ seeder ສ້າງ superadmin ສຳລັບ login ແລະທຸກ response ໃຊ້ JSON envelope ມາດຕະຖານ.”

16. ຄຳຖາມທີ່ມັກພົບໃນການສອບພາດ ແລະ ຄຳຕອບສັ້ນ

ຖາມ: ເປັນຫຍັງແຍກ controller ກັບ service?

ຕອບ: Controller ຈັດການແຕ່ HTTP; Service ເກັບກິດທຸລະກິດໃຫ້ນຳໃຊ້ຊ້າ ແລະ ທົດສອບງ່າຍ

ຖາມ: RBAC ເຮັດວຽກແນວໃດ?

ຕອບ: ແຕ່ລະ role ເກັບ permissions ເປັນ JSON. Middleware ກວດຫຼັກ `admin_module` / `user_module`. SUPER_ADMIN ຜ່ານທຸກສິດ

ຖາມ: Soft delete ແມ່ນຫຍັງ?

ຕອບ: ແຖວ admin ຖືກໝາຍລືບດ້ວຍ `is_deleted` ແລະ `deleted_at` ບໍ່ໄດ້ລືບອອກຈາກ DB ຖາວອນ

ຖາມ: ເປັນຫຍັງຕ້ອງແຊຣ໌ DB ກັບ User API?

ຕອບ: ເອກະສານກຳນົດໃຫ້ໃຊ້ຖານຂໍ້ມູນດຽວກັນ ເພື່ອໃຫ້ admin ເບິ່ງ users ຈົ່ງຈາກຕາຕະລາງ `users`

ຖາມ: ລະຫັດຜ່ານເກັບແນວໃດ?

ຕອບ: ເກັບແຕ່ bcrypt hash; ບໍ່ສົ່ງຄືນໃນ API response

ຖາມ: ຖ້າສິ່ງພົວພັນຮູ້ຈັກຈະເກີດຫຍັງ?

ຕອບ: Joi ປະຕິເສດດ້ວຍ `VALIDATION_ERR_UNKNOWN_FIELD`

17. ແຜນທີ່ໄປຣເຈັກ (ສະເພາະ Backend)

```
backend/  
|-- package.json  
|-- .env.example  
|-- .sequelizerc
```

```
`-- src/
  |-- server.js      ເລີ່ມໄປຮເຊດ
  |-- app.js         ປະກອບ Express
  |-- config/        Env + DB config
  |-- routes/        ແຜນທີ່ URL
  |-- middlewares/   Auth, RBAC, validate, errors
  |-- validations/   Joi schemas
  |-- controllers/   ຂັ້ນ HTTP
  |-- services/      Business logic
  |-- models/        Sequelize models
  |-- migrations/    ບ່ຽນໂຄງຕາຕະລາງ
  |-- seeders/       Role ເລີ່ມຕົ້ນ + superadmin
  `-- utils/         Response, JWT, password, AppError
```

Repository: <https://github.com/souksanDev1/waow-admin-task>

ຈົບຄູ່ມືໂຄງສ້າງ Backend (ພາສາລາວ)